

SCION Reference Manual

August 24, 2026

Title Spatiotemporal Clustering and Inference of Omics Networks

Version 5.0

Description Infers gene regulatory / omics networks from multi-omics data using random-forest based feature importance (a GENIE3-style approach), with optional pre-clustering of genes, permutation-based FDR thresholding of inferred edges, network visualization, and a Shiny app covering the full run / threshold / visualize workflow.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.0.0)

Imports cluster,
doParallel,
dtw,
dtwclust,
fastICA,
ggplot2,
igraph,
parallel,
randomForest,
readr,
stats,
tools,
utils

Suggests cmapR,
htmltools,
htmlwidgets,
plotly,
shiny,
shinydashboard,
shinydashboardPlus,
shinyjs,
shinytest2,
shinyWidgets,
testthat (>= 3.2.0),
visNetwork

Config/testthat/edition 3

Contents

cluster_genes	2
compute_fdr_threshold	3
compute_outdegree	4
infer_network	4
launchApp	6
load_permutations	6
permute_network	7
plot_fdr_curve	8
plot_network	9
plot_outdegree_distribution	10
plot_weight_distribution	11
read_scion_inputs	11
RS.Get.Weight.Matrix	13
run_scion	14
save_diagnostic_plots	16
save_permutation	16
write_scion_network	17
Index	18

cluster_genes	<i>Cluster genes prior to network inference</i>
---------------	---

Description

Computes a fixed gene-to-cluster assignment used by `infer_network()`. This assignment is computed once from the real, unpermuted data; when running permutations via `permute_network()`, the same assignment must be reused rather than recomputed, so that permuted networks are compared against the real network on identical gene groupings.

Usage

```
cluster_genes(
  clustering_data = NULL,
  method = c("none", "dtw", "ica", "kmeans", "upload"),
  threshold = 0.5,
  clusters_file = NULL,
  target_data = NULL,
  reg_data = NULL
)
```

Arguments

`clustering_data`
data frame or matrix of expression data to cluster on (rows = genes, columns = samples). Ignored when `method = "none"`. If `NULL` and method needs data ("dtw"/"ica"/"kmeans"), it is derived from `target_data/reg_data` – see Details.

method	clustering method: "none" (no clustering, one network for all genes), "dtw" (temporal, dynamic time warping), "ica" (non-temporal, independent component analysis), "kmeans" (non-temporal, k-means), or "upload" (use a pre-computed clusters file).
threshold	clustering threshold used by "dtw" and "ica"; ignored by "kmeans" and "upload".
clusters_file	path to a delimited text file (.csv/.tsv/.txt/.ssv, dispatched by extension) of pre-computed clusters (first column = gene names, a clusters column with cluster numbers). Required when method = "upload".
target_data, reg_data	the processed target/regulator matrices (rows = genes, columns = samples). reg_data alone is also used by method = "kmeans" to pick a starting k (see Details); target_data is only used to derive clustering_data when it isn't supplied.

Details

When clustering_data is NULL and clustering is requested, it defaults to reg_data plus whatever rows of target_data aren't already in reg_data (regulator data takes precedence for genes that are both a target and a regulator) – i.e. the same genes that will be used for network inference, combined into one matrix. This matches the historical PANOPLY behavior of auto-deriving a clustering matrix from the target/regulator data when no separate clustering file is provided.

Value

a data frame with a clusters column (row names = gene names), or NULL when method = "none".

compute_fdr_threshold *Compute a permutation-based FDR and edge-weight cutoff*

Description

Ports the validated permutation-testing procedure exactly: for each rank position in the sorted, nonzero real network weights, computes an empirical p-value from how often permuted networks exceed the real weight at that rank, BH-adjusts across ranks, and picks the smallest real weight for which FDR is still below target_fdr.

Usage

```
compute_fdr_threshold(real_network, permuted_networks, target_fdr = 0.05)
```

Arguments

real_network	the real network's edge table (must have a Weight column), e.g. the network element of run_scion() 's result.
permuted_networks	a list of permuted edge tables, as returned by permute_network() , produced against the same real_network.
target_fdr	the FDR threshold to select a weight cutoff at (default 0.05).

Value

a list:

curve	a data frame with one row per ranked real-network weight: weight, p_value, fdr – the input to <code>plot_fdr_curve()</code> .
threshold	the chosen edge-weight cutoff, or NA if no rank achieves target_fdr.
target_fdr	echoes the input.
thresholded_network	real_network filtered to Weight >= threshold.
permuted_weights	the rank x permutation matrix of (padded) permuted weights, useful for diagnostic plots.

compute_outdegree	<i>Compute each regulator's out-degree (number of edges) in a network</i>
-------------------	---

Description

Compute each regulator's out-degree (number of edges) in a network

Usage

```
compute_outdegree(network)
```

Arguments

network an edge table with a Regulator column.

Value

a data frame with Regulator and outdegree columns, one row per regulator that has at least one edge, sorted by decreasing out-degree.

infer_network	<i>Infer a (possibly clustered) network from target/regulator matrices</i>
---------------	--

Description

The core network-inference step of SCION, with no file I/O: given target and regulator expression matrices (genes as rows, samples as columns) and an optional, already-computed cluster assignment, infers one network per cluster (or a single network if cluster_assignment is NULL), optionally connects cluster hubs, and returns a single edge table. This function is called identically by `run_scion()` for the real network and by `permute_network()` for every permutation – it never recomputes clustering.

Usage

```
infer_network(
  target_data,
  reg_data,
  cluster_assignment = NULL,
  weightthreshold = 0,
  normalize = TRUE,
  connect_hubs = TRUE,
  num.cores = 1,
  ptm_sep = ".",
  seed = NULL,
  ...
)
```

Arguments

<code>target_data</code>	data frame/matrix, genes as rows, samples as columns.
<code>reg_data</code>	data frame/matrix, genes as rows, samples as columns.
<code>cluster_assignment</code>	optional data frame with a <code>clusters</code> column (row names = gene names), as returned by <code>cluster_genes()</code> . <code>NULL</code> means infer a single network across all genes.
<code>weightthreshold</code>	edges with weight below this value are dropped.
<code>normalize</code>	passed to <code>RS.Get.Weight.Matrix()</code> .
<code>connect_hubs</code>	if clustering, whether to additionally infer a network connecting each cluster's hub gene (highest out-degree regulator).
<code>num.cores</code>	passed to <code>RS.Get.Weight.Matrix()</code> .
<code>ptm_sep</code>	separator used to split a PTM-site regulator name into gene symbol + site (e.g. "." for SOX2.S35, "_" for MEF2C_S453s): when connecting cluster hubs and no hub gene is directly present in <code>target_data</code> 's row names, and always in the returned edge table (see <code>split_ptm_sites()</code>) so a PTM-annotated regulator and its plain-symbol target are recognized as the same gene.
<code>seed</code>	optional RNG seed set once, at the very start of this call (before clustering-aware seed draws or any inference). Without it, the result depends on the ambient global RNG state at call time – pass an explicit seed (or call <code>set.seed()</code> yourself right before calling) any time you need two calls to be comparable/reproducible.
<code>...</code>	additional arguments passed to <code>RS.Get.Weight.Matrix()</code> .

Value

a data frame with columns `Regulator`, `Interaction`, `Target`, `Weight`, or `NULL` if inference could not be run (e.g. no targets/regulators).

launchApp	<i>Launch the SCION Shiny app</i>
-----------	-----------------------------------

Description

A 3-tab app covering the full run -> permute/FDR -> visualize workflow. Requires shiny, shinydashboard, shinydashboardPlus, shinyjs, and plotly, which are Suggests (not hard dependencies) so that scripted/HPC use of the package doesn't require the Shiny stack to be installed.

Usage

```
launchApp(...)
```

Arguments

... additional arguments passed to shiny::shinyApp().

Value

a shiny.appobj, as returned by shiny::shinyApp().

load_permutations	<i>Read back permutation networks saved by save_permutation()</i>
-------------------	---

Description

Read back permutation networks saved by save_permutation()

Usage

```
load_permutations(dir = ".", prefix = "permutation")
```

Arguments

dir	directory to read from.
prefix	filename prefix used when saving (must match).

Value

a list of edge tables, named by permutation index (as a string) and ordered by increasing index – directly usable as the permuted_networks argument to [compute_fdr_threshold\(\)](#).

permute_network	<i>Generate permuted null networks for FDR thresholding</i>
-----------------	---

Description

Shuffles the target/regulator matrices and reruns `infer_network()` `n_permutations` times, reusing the SAME fixed `cluster_assignment` every time (clustering is never recomputed per permutation – see `cluster_genes()`). Ported from the validated lab procedure (`scion_data_processing.R`): each permutation `i` is seeded with `base_seed + i` before shuffling, which makes results independent of `num.cores` for the same reason the per-target-gene seeding in `RS.Get.Weight.Matrix()` does.

Usage

```
permute_network(
  target,
  reg,
  cluster_assignment = NULL,
  n_permutations = 100,
  indices = seq_len(n_permutations),
  permute_dim = c("col", "row"),
  base_seed = 0,
  num.cores = 1,
  weightthreshold = 0,
  normalize = TRUE,
  connect_hubs = TRUE,
  ptm_sep = ".",
  ...
)
```

Arguments

<code>target, reg</code>	the processed target/regulator matrices used for the real network (the <code>target/reg</code> elements of <code>run_scion()</code> 's result).
<code>cluster_assignment</code>	the fixed cluster assignment used for the real network (the <code>cluster_assignment</code> element of <code>run_scion()</code> 's result, or <code>NULL</code>).
<code>n_permutations</code>	number of permutations (typically 100). Ignored if <code>indices</code> is given explicitly.
<code>indices</code>	which permutation indices to run; each index <code>i</code> is seeded with <code>base_seed + i</code> , independent of every other index and of <code>num.cores</code> (see Details). Defaults to <code>seq_len(n_permutations)</code> , i.e. all of them in one call. Pass a single index – e.g. an HPC job array's task ID – to run just one permutation per job; see <code>save_permutation()/load_permutations()</code> for a file-based workflow built around exactly that pattern.
<code>permute_dim</code>	passed to <code>shuffle_matrix()</code> as <code>dim</code> ; "col" (default) matches the validated lab scheme.
<code>base_seed</code>	permutation <code>i</code> uses seed <code>base_seed + i</code> . Default 0 reproduces the lab scheme (<code>set.seed(i)</code>).
<code>num.cores</code>	when > 2 , the requested indices run across a FORK cluster (one permutation per worker at a time); each permutation's own <code>infer_network()</code> call is then forced

to `num.cores = 1` internally to avoid nesting parallelism. When `num.cores <= 2`, indices run serially (in this process or, for a single index, as whatever single HPC job called this) and their own `num.cores` is passed straight through to each `infer_network()` call.

`weightthreshold`

passed to `infer_network()`. **Must match whatever was used for the real network** (see `normalize` below), and should generally be 0: the FDR comparison in `compute_fdr_threshold()` needs the full, unthresholded network on both sides – a nonzero value applies the same manual cutoff before that comparison ever happens, biasing which edges get compared (see `run_scion()`, which enforces this automatically when calling this function with `permute = TRUE`).

`normalize`

passed to `infer_network()`. **Must match whatever was used for the real network** this permutation set will be compared against in `compute_fdr_threshold()`, and should generally be `FALSE` for both: normalizing rescales each network independently to `[0, 1]`, forcing every permutation's top edge weight to exactly 1 regardless of its actual signal, which invalidates the rank-based FDR comparison (see `run_scion()`, which enforces this automatically when calling this function with `permute = TRUE`).

`connect_hubs, ptm_sep`

passed to `infer_network()`.

...

additional arguments passed to `infer_network()`.

Details

Emits a "SCION_STAGE: permutation testing progress <done>/<total>" message after every completed permutation (serial path) or every completed batch of `num.cores - 1` permutations (parallel path – a single `parLapply()` call blocks until it returns, so there's no way to observe progress *within* one call; splitting the work into per-worker-sized batches trades a little dispatch overhead for a progress update roughly every `num.cores - 1` permutations). The Shiny app's Run tab listens for these to drive its progress bar smoothly across the whole permutation phase instead of sitting still until it's all done.

Value

a list the same length as `indices`, each element an edge table as returned by `infer_network()`, named by its permutation index (as a string).

plot_fdr_curve

Plot diagnostics from a permutation-based FDR result

Description

Plot diagnostics from a permutation-based FDR result

Usage

```
plot_fdr_curve(
  fdr_result,
  type = c("curve", "fdr_hist", "weight_comparison"),
  permutation_index = 1,
```



```

    title = NULL,
    threshold = fdr_result$threshold,
    show_target_fdr_line = TRUE,
    label_lines = FALSE
  )

```

Arguments

fdr_result	the output of <code>compute_fdr_threshold()</code> .
type	"curve" (default; weight vs. FDR scatter with the chosen threshold marked), "fdr_hist" (histogram of FDR values across ranks), or "weight_comparison" (real vs. one permuted network's weight distribution overlay).
permutation_index	which permuted network's weights to show for type = "weight_comparison" (default 1; matches <code>permuted_networks[[1]]</code> used to build <code>fdr_result</code>).
title	optional plot title – see <code>plot_weight_distribution()</code> 's title argument.
threshold	the edge-weight cutoff to mark on type = "curve" (a dashed vertical line, labeled with its value) – defaults to <code>fdr_result\$threshold</code> , but the caller can pass a different value (e.g. a manual cutoff that overrides the FDR-based one) so the marked line always reflects whichever cutoff is actually in effect. NULL/NA draws no line.
show_target_fdr_line	if TRUE (default), also draws a dashed horizontal line at <code>fdr_result\$target_fdr</code> . Set FALSE when threshold is a manual override, since the FDR target is no longer the criterion that produced it and showing it would be misleading.
label_lines	if FALSE (default), the threshold/target-FDR lines carry their value as plotly hover text only (via <code>plotly::ggplotly()</code>) – appropriate for the interactive app, where hovering is how you'd read a value off any other point on the plot too. If TRUE, also draws the value as permanent on-plot text, for contexts with no hover available (the static PDF export).

Value

a ggplot2 object.

plot_network	<i>Plot a SCION network</i>
--------------	-----------------------------

Description

A quick in-R sanity-check visualization – for anything beyond a few dozen nodes, or for a publication-quality figure, importing the edge table into Cytoscape (see `write_scion_network()`) will still give better layout control. Both the static and interactive versions use the same visual encoding: regulators are blue squares, targets are orange circles, and edge width is proportional to weight.

Usage

```
plot_network(edge_table, interactive = FALSE, legend = TRUE, ...)
```

Arguments

edge_table	an edge table with Regulator, Target, Weight columns (e.g. the network element of run_scion() 's result, or a compute_fdr_threshold() result's thresholded_network).
interactive	if FALSE (default), plots via base igraph plotting (static). If TRUE, returns an interactive visNetwork HTML widget (requires the optional visNetwork package) with on-canvas zoom/pan controls.
legend	if TRUE (default) and interactive = TRUE, attaches the HTML legend to the returned widget via <code>htmlwidgets::prependContent()</code> . Only takes effect for standalone use (R Markdown, <code>htmlwidgets::saveWidget()</code>) – Shiny's <code>renderVisNetwork()</code> doesn't transmit prepended content to the client at all (and warns about it), so the Shiny app calls this with <code>legend = FALSE</code> and renders network_legend_html() as its own separate UI element instead (see <code>tab_visualize.R</code>). Ignored for <code>interactive = FALSE</code> , which always draws its own legend directly on the plot.
...	additional arguments passed to <code>igraph::plot.igraph()</code> (static) or <code>visNetwork::visNetwork()</code> (interactive).

Value

for `interactive = FALSE`, invisibly returns the igraph object (after plotting it, with a legend, as a side effect); for `interactive = TRUE`, a visNetwork htmlwidget.

plot_outdegree_distribution

Plot the distribution of regulator out-degrees in a network

Description

Plot the distribution of regulator out-degrees in a network

Usage

```
plot_outdegree_distribution(network, bins = 30, title = NULL)
```

Arguments

network	an edge table with a Regulator column.
bins	number of histogram bins.
title	optional plot title – see plot_weight_distribution() 's title argument.

Value

a ggplot2 object.

plot_weight_distribution

Plot the distribution of edge weights in a network

Description

Plot the distribution of edge weights in a network

Usage

```
plot_weight_distribution(
  network,
  bins = 100,
  cutoff = NULL,
  title = NULL,
  label_line = FALSE
)
```

Arguments

network	an edge table with a Weight column, e.g. the network element of <code>run_scion()</code> 's result. Always pass the full, unthresholded network here (not an already-filtered one) so cutoff is visible in context against the whole distribution.
bins	number of histogram bins.
cutoff	optional edge-weight cutoff (FDR-based or manual) to mark with a dashed vertical line. NULL (default) or NA draws no line.
title	optional plot title – NULL (default) omits it. The interactive app view skips this (its box header already names the plot); <code>save_diagnostic_plots()</code> and the app's PDF export set one, since an exported plot has no other title to identify it by.
label_line	if FALSE (default), the cutoff line carries its value as plotly hover text only (via <code>plotly::ggplotly()</code>) – appropriate for the interactive app. If TRUE, also draws the value as permanent on-plot text, for contexts with no hover available (the static PDF export).

Value

a ggplot2 object.

read_scion_inputs

Read SCION input matrices from a delimited text format or GCT

Description

Reads target and regulator expression matrices (and, optionally, a separate clustering matrix), restricts each to a gene list if provided, and drops any row containing a missing value – SCION does not support missing values, so unlike the private-version `na.max` proportional filter, this is always an all-or-nothing drop.

Usage

```
read_scion_inputs(
  target_data_file,
  reg_data_file,
  target_genes_file = NULL,
  reg_genes_file = NULL,
  gene_list_header = TRUE,
  clustering_data_file = NULL,
  format = c("auto", "delimited", "gct")
)
```

Arguments

`target_data_file`, `reg_data_file`

path to target/regulator expression matrices. Delimited text (first column = gene names, remaining columns = samples): .csv (comma), .tsv/.txt (tab), or .ssv (semicolon), dispatched by extension. GCT: a GCT(x) file, read via `cmapR::parse_gctx()` (requires the optional `cmapR` package – install with `BiocManager::install()`).

`target_genes_file`, `reg_genes_file`

optional path to a delimited text file (.csv/.tsv/.txt/.ssv, same extension dispatch as above) listing which genes to keep as targets/regulators (first column = gene names). NULL (default) keeps every gene present in the corresponding data file.

`gene_list_header`

whether `target_genes_file/reg_genes_file` have a header row. Default TRUE (matches this package's own tutorial data). Set to FALSE for a plain one-gene-symbol-per-line file with no header (e.g. PANOPLY's `TF_file` convention) – with the default TRUE, such a file would silently have its first gene misread as a column header and dropped.

`clustering_data_file`

optional path to a delimited text clustering matrix (rows = genes, columns = samples; same extension dispatch as above), used by `cluster_genes()`. Restricted to genes that appear in `target_genes_file` or `reg_genes_file` when those are given.

`format`

"auto" (default), "delimited", or "gct". "auto" detects GCT(x) from `target_data_file`'s extension (.gct/.gctx) and falls back to delimited text otherwise – pass "delimited"/"gct" explicitly to override. Regulator gene names may use either a dot (SOX2.S35) or underscore (MEF2C_S453s) PTM-site convention; both are left as-is here (see the `ptm_sep` argument of `infer_network()` for where the convention matters downstream).

Value

a list with `target` and `reg` data frames (genes as rows, samples as columns, row names made syntactically valid via `make.names()`), and `cluster_data` (or NULL if `clustering_data_file` was not given).

RS.Get.Weight.Matrix	<i>Infer a weighted regulator -> target network via random-forest importance</i>
----------------------	---

Description

For each target gene, fits a random forest predicting its expression from all regulator features, and records each regulator's importance as the edge weight. This is the GENIE3-style core of SCION's network inference, and is called identically for the real network and for every permutation generated by [permute_network\(\)](#).

Usage

```
RS.Get.Weight.Matrix(
  target.matrix,
  input.matrix,
  K = "sqrt",
  nb.trees = 10000,
  importance.measure = "%IncMSE",
  seed = NULL,
  trace = TRUE,
  normalize = TRUE,
  num.cores = 1,
  ...
)
```

Arguments

target.matrix	data frame/matrix of target expression, samples as rows, genes as columns.
input.matrix	data frame/matrix of regulator (input) expression, samples as rows, genes as columns.
K	number of candidate regulators considered at each tree split: "sqrt", "all", or an integer.
nb.trees	number of trees per random forest.
importance.measure	"IncNodePurity" or "%IncMSE".
seed	optional RNG seed for the top-level per-target seed draw. Must be set consistently between a real network and its permutations for the permutation seeding scheme in permute_network() to be reproducible.
trace	if TRUE, emit a progress message per target gene.
normalize	if TRUE, rescale the weight matrix to [0, 1].
num.cores	number of cores to use: a num.cores - 1 FORK cluster is used to parallelize across target genes (disabled when num.cores <= 2).
...	additional arguments passed to <code>randomForest::randomForest()</code> .

Value

a numeric matrix of edge weights (targets x regulators), or NULL if there are no targets or no regulators.

run_scion

*Run SCION: read inputs, cluster once, infer a network***Description**

User-facing wrapper around `read_scion_inputs()`, `cluster_genes()`, and `infer_network()`. Unlike the legacy `SCION()` function, this does not require a working directory or hardcoded file output – it returns everything in memory, and optionally writes a Cytoscape-importable edge table if `output_file` is given. The returned list (particularly `target`, `reg`, and `cluster_assignment`) has everything `permute_network()` needs to run permutations against this same network without recomputing clustering.

Usage

```
run_scion(
  target_data_file,
  reg_data_file,
  target_genes_file = NULL,
  reg_genes_file = NULL,
  gene_list_header = TRUE,
  format = c("auto", "delimited", "gct"),
  clustering_method = c("none", "dtw", "ica", "kmeans", "upload"),
  clustering_data_file = NULL,
  clustering_threshold = 0.5,
  clusters_file = NULL,
  connect_hubs = TRUE,
  weightthreshold = 0,
  normalize = TRUE,
  num.cores = 1,
  ptm_sep = ".",
  seed = 2020,
  permute = FALSE,
  n_permutations = 100,
  permute_dim = c("col", "row"),
  base_seed = 0,
  target_fdr = 0.05,
  output_file = NULL,
  ...
)
```

Arguments

`target_data_file`, `reg_data_file`, `target_genes_file`, `reg_genes_file`
`gene_list_header`, `format` passed to `read_scion_inputs()`.

`clustering_method`
 passed to `cluster_genes()` as method: "none" (default), "dtw", "ica", "kmeans", or "upload".

`clustering_data_file`
 path to a clustering matrix, required unless `clustering_method` is "none" or "upload".

clustering_threshold	passed to <code>cluster_genes()</code> as threshold.
clusters_file	passed to <code>cluster_genes()</code> as clusters_file, required when clustering_method = "upload".
connect_hubs, num.cores, ptm_sep	passed to <code>infer_network()</code> .
weightthreshold	passed to <code>infer_network()</code> . Forced to 0 (with a warning) whenever permute = TRUE, regardless of what's passed – the FDR comparison needs the full, un-thresholded network on both sides; apply a cutoff to the result afterward instead (see <code>compute_fdr_threshold()</code>).
normalize	passed to <code>infer_network()</code> . Forced to FALSE (with a warning) whenever permute = TRUE, regardless of what's passed – normalizing rescales each network (real and every permutation) independently to [0, 1], which would force every permutation's top edge weight to exactly 1 and invalidate the rank-based FDR comparison.
seed	RNG seed set once, before clustering and inference, so the same inputs produce the same network every run. Default matches the legacy SCION() behavior. Set to NULL to skip seeding.
permute	if TRUE, also run <code>permute_network()</code> and <code>compute_fdr_threshold()</code> against the just-inferred network, in this same call. Runs n_permutations permutations using the SAME weightthreshold, normalize, connect_hubs, ptm_sep, num.cores, and ... used for the real network above – there is no separate way to set these for the permutations, since the FDR calculation requires them to match. For sharding permutations across an HPC job array instead of running them all in this one call, use permute = FALSE here and call <code>permute_network()</code> / <code>save_permutation()</code> / <code>load_permutations()</code> directly against this call's target/reg/cluster_assignment
n_permutations, permute_dim, base_seed	passed to <code>permute_network()</code> when permute = TRUE.
target_fdr	passed to <code>compute_fdr_threshold()</code> when permute = TRUE.
output_file	optional path to write the final edge table to (tab-separated, Cytoscape-importable). NULL (default) writes nothing. When permute = TRUE, writes the FDR-thresholded network, not the raw one. Also triggers <code>save_diagnostic_plots()</code> , saved alongside it (same directory, named from output_file's base name) – there's no Shiny app to view/download them from interactively in this CLI/scripted path, so they're written automatically instead of requiring a separate call.
...	additional arguments passed to <code>infer_network()</code> / <code>RS.Get.Weight.Matrix()</code> (and, when permute = TRUE, to the internal <code>permute_network()</code> call as well, so e.g. nb.trees consistent between the real network and its permutations).

Value

a list with network (the real edge table), target, reg (the processed input matrices), cluster_assignment (or NULL), params (the arguments used, for reference / for feeding into `permute_network()` yourself), and – only when permute = TRUE – permuted_networks (the `permute_network()` result), fdr_result (the `compute_fdr_threshold()` result), and network_thresholded (shorthand for fdr_result\$thresholded_network).

save_diagnostic_plots	<i>Save SCION's diagnostic plots as publication-quality PDFs</i>
-----------------------	--

Description

Writes the edge weight distribution and regulator out-degree distribution (and, if permutations were run, the FDR curve and real-vs-permuted weight comparison) as separate single-plot PDFs via `ggplot2::ggsave()` – vector output, so resolution is never a concern. This is what `run_scion()` calls automatically whenever `output_file` is set (i.e. CLI/scripted use, where there's no app to view the interactive versions in); call it directly for any other `run_scion()` result you want plots from.

Usage

```
save_diagnostic_plots(
  result,
  dir,
  prefix = "diagnostics",
  width = 7,
  height = 5,
  units = "in"
)
```

Arguments

result	a <code>run_scion()</code> result list.
dir	directory to save into (created, recursively, if it doesn't exist).
prefix	filename prefix for each saved plot.
width, height, units	passed to <code>ggplot2::ggsave()</code> .

Value

invisibly, a character vector of the file paths written.

save_permutation	<i>Save one permutation's network to disk</i>
------------------	---

Description

Companion to `load_permutations()` for an HPC job-array workflow: each array task runs one permutation (via `permute_network(..., indices = task_id)`) and saves its result with this function; a final job reads them all back with `load_permutations()` and passes them to `compute_fdr_threshold()`.

Usage

```
save_permutation(network, index, dir = ".", prefix = "permutation")
```


Arguments

network	a single permuted network (edge table), e.g. <code>permute_network(..., indices = task_id)[[1]]</code> .
index	the permutation index this network corresponds to (e.g. the HPC job array task ID).
dir	directory to save into (created if it doesn't exist).
prefix	filename prefix; the file is saved as <code><prefix>_<index>.rds</code> .

Value

the path written to, invisibly.

write_scion_network	<i>Write a SCION network to a Cytoscape-importable file</i>
---------------------	---

Description

Write a SCION network to a Cytoscape-importable file

Usage

```
write_scion_network(network, output_file)
```

Arguments

network	an edge table as returned in the network element of <code>run_scion()</code> 's result.
output_file	path to write to (tab-separated).

Index

`cluster_genes`, [2](#)
`cluster_genes()`, [5](#), [7](#), [12](#), [14](#), [15](#)
`compute_fdr_threshold`, [3](#)
`compute_fdr_threshold()`, [6](#), [8–10](#), [15](#), [16](#)
`compute_outdegree`, [4](#)

`infer_network`, [4](#)
`infer_network()`, [2](#), [7](#), [8](#), [12](#), [14](#), [15](#)

`launchApp`, [6](#)
`load_permutations`, [6](#)
`load_permutations()`, [7](#), [15](#), [16](#)

`make.names()`, [12](#)

`network_legend_html()`, [10](#)

`permute_network`, [7](#)
`permute_network()`, [2–4](#), [13–15](#)
`plot_fdr_curve`, [8](#)
`plot_fdr_curve()`, [4](#)
`plot_network`, [9](#)
`plot_outdegree_distribution`, [10](#)
`plot_weight_distribution`, [11](#)
`plot_weight_distribution()`, [9](#), [10](#)
`plotly::ggplotly()`, [9](#), [11](#)

`read_scion_inputs`, [11](#)
`read_scion_inputs()`, [14](#)
`RS.Get.Weight.Matrix`, [13](#)
`RS.Get.Weight.Matrix()`, [5](#), [7](#), [15](#)
`run_scion`, [14](#)
`run_scion()`, [3](#), [4](#), [7](#), [8](#), [10](#), [11](#), [16](#), [17](#)

`save_diagnostic_plots`, [16](#)
`save_diagnostic_plots()`, [11](#), [15](#)
`save_permutation`, [16](#)
`save_permutation()`, [7](#), [15](#)
`shuffle_matrix()`, [7](#)
`split_ptm_sites()`, [5](#)

`write_scion_network`, [17](#)
`write_scion_network()`, [9](#)